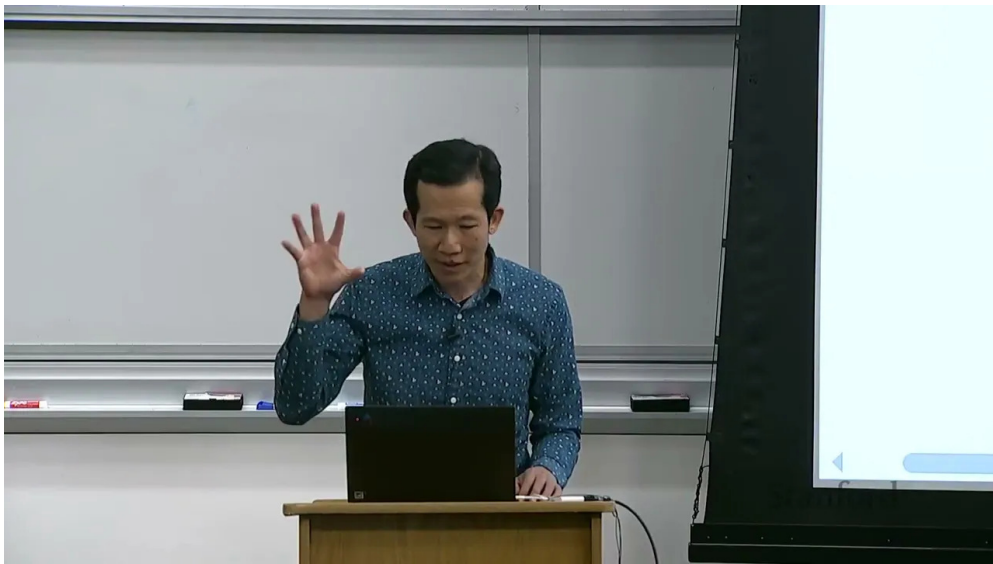


课程笔记

CS336 Lecture 2: Building a Model in PyTorch

基于 Tatsu Hashimoto 授课内容整理

2025 年 4 月



视频作者/频道: Stanford Online

发布日期: 2025-04

视频时长: 1:19:18

视频链接: 未填写

目录

1 引言：从概览走向实现	2
1.1 一个先算清楚的 napkin math	2
2 Memory Accounting	3
2.1 张量是深度学习的基本单位	3
2.2 常见浮点类型	3
2.3 CPU 与 GPU 的内存位置	4
2.4 Tensor storage, stride 和 view	4
2.5 切片、转置和掩码	5
2.6 逐元素运算和矩阵乘法	5
2.7 Einops 和维度命名	5
3 Compute Accounting	6
3.1 FLOPs 和 FLOP/s	6
3.2 线性模型的 FLOPs	6
3.3 更一般的近似	6
3.4 模型 FLOPs 利用率 MFU	7
3.5 反向传播的 FLOPs	7
4 Models	7
4.1 <code>nn.Parameter</code>	7
4.2 初始化：为什么要按输入维度缩放	7
4.3 自定义模块	8
5 Training Loop and Best Practices	8
5.1 随机性与可复现	8
5.2 数据加载：把序列切成 batch	9
5.3 Pinned memory 和异步拷贝	9
5.4 优化器	9
5.5 训练时的内存核算	10
5.6 训练循环	10
5.7 Checkpointing	11
5.8 混合精度训练	11
6 本讲总结	11

1 引言：从概览走向实现

上一讲先概览了语言模型、tokenization 以及从零构建模型的动机。本讲进入真正的实现层面，目标不是再讲 Transformer 的整体结构，而是把训练模型所需要的 **PyTorch 原语** 一层层搭起来。

本讲的三件事

1. 从张量开始，搞清楚模型参数、梯度、优化器状态和数据到底如何占用内存。
2. 从矩阵乘法开始，学会对 FLOPs 和 FLOP/s 做资源核算。
3. 从 ‘nn.Parameter’、‘nn.Module’、优化器和训练循环出发，搭出一个最小但完整的训练系统。

讲义里的核心方法很朴素：**先算账，再写代码**。如果不知道一个张量有多少字节、一次矩阵乘法有多少 FLOPs、一次训练步会额外产生多少梯度和优化器状态，就很难判断模型为什么慢，也很难知道什么时候该换数据类型、换实现方式或者换训练策略。

两类资源

本讲只盯住两种资源：

- **内存**：参数、梯度、激活值、优化器状态都要装得下。
- **计算**：训练和推理的成本通常都能归结成 FLOPs。

1.1 一个先算清楚的 napkin math

先看两个典型估算。

1. 训练一个 70B 参数模型，在 15T tokens 上，用 1024 张 H100，大概需要多久？
2. 如果只看显存，8 张 80GB H100 上，使用朴素 AdamW，最多能训练多大的 dense 模型？

第一题里，密集 Transformer 的训练 FLOPs 可以粗略记为

$$6 \times (\text{参数量}) \times (\text{token 数})$$

于是总 FLOPs 约为

$$6 \cdot 70 \times 10^9 \cdot 15 \times 10^{12}.$$

如果把 H100 的有效算力设成峰值的一半，再乘上 1024 张卡，一天能提供多少 FLOPs 也就能算出来，最后得到大约 **144 天**。这个估算很粗，但足够说明规模。

第二题里，H100 显存是 80GB。若朴素地用 float32 存参数、梯度和 AdamW 的一阶/二阶矩估计，那么每个参数大约需要

$$4 (\text{参数}) + 4 (\text{梯度}) + 4 (\text{一阶矩}) + 4 (\text{二阶矩}) = 16 \text{ bytes}.$$

8 张卡总共约 640GB，因此最多支持大约

$$\frac{640 \text{ GB}}{16 \text{ bytes}} \approx 40\text{B parameters}.$$

这还没有算激活值，所以只是上界。

为什么要算这个

当数字开始变大以后，FLOPs 和内存都会直接变成钱。训练系统不是“能跑就行”，而是“能以可接受的成本跑”。这就是资源核算的意义。

2 Memory Accounting

2.1 张量是深度学习的基本单位

在 PyTorch 里，几乎所有东西最终都是张量：

- 模型参数
- 梯度
- 优化器状态
- 数据
- 激活值

张量的两件事

一个 tensor 的内存成本只看两件事：

- 元素个数 `numel()`
- 每个元素占多少字节 `element_size()`

因此最简单的内存估计就是

$$\text{memory} = \text{numel} \times \text{element size}.$$

2.2 常见浮点类型

类型	字节数	范围/特点	训练时常见用途
float32	4	范围大，精度高	参数、梯度、主权重
float16	2	省内存，但动态范围小	前向激活、部分训练
bfloat16	2	范围接近 float32，精度略低	训练和推理中的常用折中
fp8	1	更激进的压缩	高性能训练/推理的实验方向

表 1: 常见数值类型的内存与训练特性

这里最重要的结论是：**bfloat16 不是“更小的 float32”，而是“保留了 float32 的指数范围，但牺牲一些尾数精度”**。在深度学习里，这种取舍通常比 float16 更稳。

为什么训练比推理更难低精度化

训练时会同时涉及参数、梯度、优化器状态和反向传播链式法则。数值误差会在更新过程中累积，因此训练对数值稳定性更敏感；推理阶段没有反向传播，通常可以更激进地量化。

2.3 CPU 与 GPU 的内存位置

默认情况下，PyTorch tensor 存在 CPU 内存中。要利用 GPU 的并行计算能力，必须显式把 tensor 迁移到 GPU 显存中。

位置	特点	常见操作
CPU paged memory	默认、可分页、适合常规处理	数据预处理、批生成
CPU pinned memory	页锁定，可异步拷贝到 GPU	数据加载器预取
GPU memory	计算速度最快、容量最宝贵	参数、激活、临时计算结果

表 2: 训练中常见的内存位置

Pinned memory 的意义

把 CPU 上的 batch 放到 pinned memory 后，可以用 `non_blocking=True` 异步拷贝到 GPU。这样可以把“CPU 取下一批数据”和“GPU 处理当前批数据”重叠起来。

2.4 Tensor storage, stride 和 view

PyTorch tensor 不只是“值的集合”，它更像是：

- 一块连续或者非连续的底层存储；
- 加上一组元数据，告诉你每个维度怎么映射到存储地址；
- 以及形状、dtype、device 等信息。

在二维张量里，‘stride(0)’表示跨行要跳多少元素，‘stride(1)’表示跨列要跳多少元素。对于一个标准行主序矩阵来说，行 stride 通常等于列数，列 stride 通常为 1。

view 是免费的，copy 不是

‘view’、切片和转置很多时候都只是不同的“视图”，不会复制底层数据，因此几乎不耗额外内存。真正触发 ‘contiguous()’ 或显式拷贝时，才会同时增加内存和计算成本。

一个容易踩坑的点是：不是所有视图都能再 ‘view’ 成任意形状。若张量是非连续的，直接 ‘view’ 常常会报错，这时要先调用 ‘contiguous()’。

为什么这一点重要

很多模型代码看起来只是“换了个形状”，实际上却偷偷复制了一大块张量。只要复制发生得不该发生，内存就会涨，性能也会掉。

2.5 切片、转置和掩码

切片、列选择、转置等操作通常只返回新的视图而不是复制数据。对于 causal attention 这类场景，‘triu’ 很常见，因为它可以快速生成上三角矩阵，用来屏蔽未来 token 的信息。

causal mask 的直觉

如果 $M[i, j] = 1$ 表示位置 i 对位置 j 有贡献，那么自回归语言模型要求未来位置不能泄露到过去，于是只保留上三角或下三角的合法连接。

2.6 逐元素运算和矩阵乘法

逐元素运算包括加法、乘法、平方、开方、‘triu’ 等。这类操作一般是线性的，FLOPs 规模约为 $O(mn)$ 。

真正的深度学习主角还是矩阵乘法：

$$X \in \mathbb{R}^{B \times D}, \quad W \in \mathbb{R}^{D \times K}, \quad Y = XW \in \mathbb{R}^{B \times K}.$$

这里的 FLOPs 约为

$$2BDK,$$

因为每个输出元素都需要一次乘法和一次加法。

运算	复杂度	备注
逐元素加/乘	$O(mn)$	规模线性
矩阵乘法	$O(mnp)$	通常是计算瓶颈
反向传播	约为前向的 2 倍到 3 倍	依赖具体图结构

表 3: 训练里最常见的计算量级

当输入张量多出 batch 维和 sequence 维时，PyTorch 的矩阵乘法会自动在前面的维度上广播并批处理。也就是说，只要最后两维满足矩阵乘法规则，前面的维度会被视作并行的“外层循环”。

2.7 Einops 和维度命名

原生 PyTorch 容易在 ‘-2’、‘-1’ 这种负索引里把维度写乱。‘einops’ 的价值在于把维度变成可读的名字。

维度命名的价值

- ‘batch seq hidden’ 一眼就知道张量语义。
- ‘einsum’ 可以把广播和求和写得更清楚。
- ‘reduce’ 和 ‘rearrange’ 让 reshape 逻辑更不容易出错。

这讲里最常见的三种写法是：

- ‘einsum(x, y, "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")’
- ‘reduce(x, "... hidden -> ...", "sum")’

- ‘rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)’

3 Compute Accounting

3.1 FLOPs 和 FLOP/s

FLOP 是一次浮点操作，比如一次加法或乘法。FLOP/s 则是硬件每秒能执行多少浮点操作。这里要避免两个缩写混淆：

- **FLOPs**：算了多少浮点操作。
- **FLOP/s**：每秒能做多浮点操作。

资源核算的核心问题

训练时间约等于

$$\frac{\text{总 FLOPs}}{\text{有效 FLOP/s}}.$$

所以除了算法本身，我们还必须知道硬件的有效利用率。

3.2 线性模型的 FLOPs

先从最简单的线性模型开始。若有 B 个样本、每个样本维度为 D 、输出维度为 K ，那么前向传播的矩阵乘法

$$Y = XW$$

大约需要

$$2BDK$$

个 FLOPs。

后向传播则需要对参数和中间激活分别做链式法则，整体大约是前向的两倍，所以线性层的完整训练步常用近似是：

$$\text{forward} + \text{backward} \approx 6BDK.$$

为什么是 6

粗略地看：

- 前向：一次乘法 + 一次加法，约 $2BDK$
- 反向：对参数和输入各再做一次类似的乘加，约 $4BDK$

合起来就是约 $6BDK$ 。

3.3 更一般的近似

对于 Transformer 这类模型，虽然结构更复杂，但一阶近似下，训练 FLOPs 仍可以写成：

$$6 \times (\text{token 数}) \times (\text{参数量}).$$

这也是前面 napkin math 的来源。

3.4 模型 FLOPs 利用率 MFU

硬件规格表给的是“峰值”性能，但真实训练不可能一直跑满。于是引入：

$$\text{MFU} = \frac{\text{actual FLOP/s}}{\text{promised FLOP/s}}.$$

MFU 不是一个抽象学术指标，而是直接决定你有多少硬件预算被真正转化成训练速度。一般来说，MFU 大于 0.5 已经算不错，尤其当矩阵乘法占主导时更是如此。

为什么 float32 和 bfloat16 的 FLOP/s 不一样

硬件规格里的“峰值 FLOP/s”强依赖数据类型。很多 GPU 对 bfloat16/float16 的吞吐量远高于 float32，所以同一块卡上，切换 dtype 会显著改变你的实际吞吐。

3.5 反向传播的 FLOPs

以两层线性模型为例：

$$x \xrightarrow{w_1} h_1 \xrightarrow{w_2} h_2 \rightarrow \text{loss}.$$

前向已经需要两次矩阵乘法。反向传播时，对 w_2 、 h_1 、 w_1 以及输入的梯度分别做链式法则，整体 FLOPs 约为前向的两倍。因此整个训练步常见的经验法则仍然是：

$$\text{train step FLOPs} \approx 6 \times (\text{tokens}) \times (\text{parameters}).$$

这个近似什么时候够用

只要你在做的是规模估算、预算估算、训练时长估算，这个近似就足够好。它不是为了证明定理，而是为了让你能快速判断“这个实验值不值得跑”。

4 Models

4.1 nn.Parameter

PyTorch 里模型参数通常以 ‘nn.Parameter’ 的形式保存。它和普通 tensor 类似，但会自动被 ‘nn.Module’ 注册为可训练参数，并出现在 ‘parameters()’ 和 ‘state_dict()’ 里。

参数与普通 tensor 的差异

‘nn.Parameter’ 的本质还是 tensor，但它明确表达了“这是要被训练的东西”。这会影响：

- 自动求导
- 模型参数遍历
- checkpoint 保存/恢复

4.2 初始化：为什么要按输入维度缩放

如果直接用高斯随机初始化，输出的尺度会随着输入维度增长而变大。一个简单的做法是除以 $\sqrt{\text{input_dim}}$ ，这样可以让输出的方差大致稳定。

Xavier 的直觉

初始化要尽量让前向传播和反向传播中的数值尺度都保持稳定。最简单的经验就是：参数规模要随输入维度做归一化。

在更实际的实现里，通常会使用 truncated normal，把极端离群值裁掉，进一步减少训练不稳定的风险。

4.3 自定义模块

下面这类简单线性模块很关键，因为它展示了 ‘nn.Module’ 的最小骨架。

```
1 class Linear(nn.Module):
2     def __init__(self, input_dim: int, output_dim: int):
3         super().__init__()
4         self.weight = nn.Parameter(
5             torch.randn(input_dim, output_dim) / np.sqrt(input_dim)
6         )
7
8     def forward(self, x: torch.Tensor) -> torch.Tensor:
9         return x @ self.weight
```

再往上堆叠多个层，就能得到一个简单的深度线性模型。重点不是“线性模型很强”，而是“理解模块化结构如何把参数组织起来”。

state_dict 里有什么

state_dict() 负责保存模型的状态。对一个模块来说，它通常包括：

- 所有 ‘nn.Parameter’
- 有状态层的缓存

所以 checkpoint 不只是保存参数本身，而是保存整个训练状态。

5 Training Loop and Best Practices

5.1 随机性与可复现

训练里随机性来自很多地方：

- 参数初始化
- dropout
- 数据顺序

因此调试时最好把三套随机种子一次性设好：

- torch.manual_seed
- numpy.random.seed

- `random.seed`

为什么要统一随机种子

当实验结果不稳定时，最怕的是“同一个 bug 在不同 run 里表现不同”。先锁定随机性，才能把问题缩小到实现本身。

5.2 数据加载：把序列切成 batch

语言模型的数据本质上是一串整数 token。最常见的做法是把它序列化到 numpy 数组，再用‘`memmap`’懒加载。

```
1 start_indices = torch.randint(len(data) - sequence_length, (batch_size,))
2 x = torch.tensor([data[start:start + sequence_length] for start in start_indices])
```

这段逻辑的语义很直接：

- 随机采样若干起点；
- 从每个起点切出一段固定长度的 token 序列；
- 堆成一个 batch。

为什么要用 memmap

真实数据集可能非常大，不能一次性全读进内存。‘`np.memmap`’允许只把当前访问到的部分映射到内存中，因此更适合大规模语言建模。

5.3 Pinned memory 和异步拷贝

若 batch 先放在 pinned memory，再执行

```
x.to(device, non_blocking=True),
```

CPU 到 GPU 的拷贝可以异步进行。这样就能把数据搬运和 GPU 计算重叠起来，减少空转。

数据管线也是性能瓶颈

很多时候模型不慢，是因为数据喂不够快。只盯着算子优化而不看数据加载，常常会把训练吞吐的真正瓶颈漏掉。

5.4 优化器

本讲先自己实现了两个最简单的优化器：

- SGD
- AdaGrad

然后再把它们和更常见的优化器对应起来：

- Momentum = SGD + 指数滑动平均

- AdaGrad = SGD + 累积平方梯度
- RMSProp = AdaGrad + 指数滑动平均的平方梯度
- Adam = RMSProp + Momentum

优化器的代价

优化器不仅改变更新规则，还会显式增加状态量。比如 Adam 至少要保存两份额外状态，这就是为什么优化器内存常常比你想象得更贵。

5.5 训练时的内存核算

对一个普通的前馈模型，训练时要同时考虑：

部分	数量级	说明
参数	N_p	模型权重
激活值	N_a	前向过程中保存的中间结果
梯度	N_p	每个参数对应一个梯度
优化器状态	N_p 或更多	取决于算法

表 4: 训练步的常见内存组成

若用 float32 朴素训练，则总内存可以粗略写成：

$$4 \times (N_p + N_a + N_p + N_p) = 4(3N_p + N_a).$$

这也是为什么“模型参数看起来装得下”，但“训练时还是爆显存”。

5.6 训练循环

标准训练循环可以压缩成四步：

1. 前向：算预测值和 loss
2. 反向：‘loss.backward()’
3. 更新：‘optimizer.step()’
4. 清零：optimizer.zero_grad(set_to_none=True)

```

1 for t in range(num_train_steps):
2     x, y = get_batch(B=B)
3     pred_y = model(x)
4     loss = F.mse_loss(pred_y, y)
5     loss.backward()
6     optimizer.step()
7     optimizer.zero_grad(set_to_none=True)

```

训练循环的本质

训练循环不是“写起来麻烦”，而是“把前向、反向、更新三种状态转换显式地串起来”。一旦逻辑清楚了，调参和定位 bug 都会容易很多。

5.7 Checkpointing

训练语言模型往往很久，而且很容易中断。所以要定期保存 checkpoint。

- `model.state_dict()`
- `optimizer.state_dict()`

这样不仅能恢复参数，还能恢复优化器状态，避免“恢复后训练轨迹明显变了”。

只存模型不够

如果只存 model，不存 optimizer，那么恢复后学习率状态、动量累积、平方梯度缓存都没了，训练行为会和之前不一样。

5.8 混合精度训练

训练和推理都受数值精度影响，但训练更敏感。因此常见策略是：

- 前向和激活尽量用 bfloat16 或 fp8；
- 参数、梯度和关键状态保留 float32；
- 必要时用 AMP 自动处理类型切换。

训练和推理的差异

推理阶段你可以更激进地量化，因为没有反向传播；训练阶段必须在稳定性和吞吐之间找平衡。这也是为什么“低精度训练”比“低精度推理”难得多。

6 本讲总结

本讲把“训练一个模型”拆成了可操作的最小单元：

- 张量如何占内存；
- 矩阵乘法如何计算 FLOPs；
- 参数如何封装进 ‘nn.Module’；
- 数据如何切 batch；
- 优化器、训练循环、checkpoint 和混合精度如何协同工作。

最后要记住的一句话

深度学习系统不是“把模型写出来”就结束了，而是要持续回答三个问题：

1. 这个东西占多少内存？
2. 这个东西消耗多少 FLOPs？
3. 这个东西能不能稳定地、可复现地训练起来？

做完 Assignment 1 之后，这些概念会变得更具体，也会更牢固，因为你会亲手把它们用在真正的 Transformer 上。